

Grow and Refine: Co-Evolving Memory for Long-Horizon Engineering Agents

Qixin Xu

Engineering Task

- Long horizon, dense feedback:

- 10+ turns, each turn has reasoning and code snippet
- Every turn receives external feedback

→ a stream of experience to store & reuse, How to use memory?

- A bag of tricks:

- Human experts >> Sota Agents
 - Human experts: many task-specific tricks
 - Agents: few general methods

→ close the gap by accumulating tricks, How to grow memory?

Motivation: Memory should be a flywheel, not a pile

A. How to use memory?

- a. **trajectory-level:** one block at start, static, no per-action reconsider
- b. **turn-level:** re-retrieve per turn, action-conditioned, re-judge each turn

B. How to grow memory?

- a. **append-only:** bullet per lesson, unbounded, contradictions
- b. **Rewrite-only:** edit in place, loses past, no trace
- c. **dual-update:**
 - i. memory **grows + self-corrects**
 - ii. Delta-append new insights
 - iii. caveat scopes over-generalized entries

Method: Per-turn, action-conditioned retrieval

Memory Use

- Three Agent actions: draft / improve / debug
- How action shapes memory **writes**
 - Write only on success: valid draft / score-improving improve / successful debug
 - only task-relevant evidence enters; failed tries don't pollute the pool
- How action shapes memory **reads** **【Pending for further exp】**
 - Each action retrieves only its own past successes
 - (**draft**↔valid draft, **improve**↔score-improving improve, **debug**↔successful debug)

Method: Grow and Refine

Memory Grow

After round t (Agent + memory v_t):

Memory update:

- Grow (Δ -append)
 - scan round- t trajectories, keep the wins (valid draft / improving improve / debug-fix)
 - each win $\rightarrow$ a new entry

$\rightarrow$ memory **covers more strategy space**

- Refine (caveat)
 - find regressions (Agent + memory $v_t < \text{Agent}$)
 - trace the misapplied entry $\rightarrow$ attach a scope caveat

$\rightarrow$ memory **refines the boundary** of each insight

Return memory v_{t+1} and enter round $t+1$

Method: Co-Evolving Memory across Multiple Agents

- Shared memory, heterogeneous writers
- Group continual learning across rounds
 - Read: all agents query shared memory v_t
 - Write:
 - each agent's wins $\rightarrow$ new entries
 - regressions $\rightarrow$ caveats (same pool)
 - Memory v_{t+1} = union:
 - one model's mistake $\rightarrow$ everyone's caveat
 - one model's win $\rightarrow$ everyone's insight

Results: Setup

- **Benchmark:** 18 real Kaggle competitions from **DSGym**
- **Metric:** Kaggle public leaderboard percentile
- **Backbones:** claude sonnet 4.6, GPT-5.2, Gemini 3 flash
- **Baselines and Ablations:**
 - static memory, no flywheel ✓
 - Δ -only update ✓
 - Caveat-only update ✓
 - Trajectory-level memory ✓
 - Single-model vs. multi-model
 - Update on the fly
 - Other baseline, ACE ✓
 - Different retrieval method ✓

Results: main results

	Algorithm	DSGym-Easy	DSGym-hard	DSGym-hard Medals
Claude-sonnet 4.6	—	46.4	35.3	—
GPT-5.2	—	43.6	31.3	—
Gemini 3 Flash	—	52.1	31.8	
Claude-sonnet 4.6	Grow & Refine	64.7	52.6	 
GPT-5.2	Grow & Refine	57.3	43.3	 
Gemini 3 Flash	Grow & Refine	53.9	40.2	 

- Three model improves and earn more medals

Results: memory update method【三个折线】

Results: turn-level memory vs. trajectory-level memory

	Memory use	DSGym-Easy	DSGym-hard
Claude-sonnet 4.6	Trajectory-level memory	45.3	31.0
	Turn-level memory	48.8	42.2
GPT-5.2	Trajectory-level memory	54.4	26.7
	Turn-level memory	57.2	39.2

- turn-level memory > trajectory level memory
- performance gap: Hard split > Easy split

Results: Group wisdom vs. single model knowledge

Results: Retriever Sensitivity

	Retriever	DSGym-Easy	DSGym-hard
Claude-sonnet 4.6	—	46.4	35.3
	BM25	59.5	44.2
	text-embedding-3-small	61.8	47.4
	text-embedding-3-large	63.1	48.9
	Qwen3-Embedding-8B	62.0	46.8

- Different retrievers consistently improve
- Better retriever, better scores